



كلية تكنولوجيا الصناعة والطاقة

Information Technology Department



جامعه برج العرب التكنولوجيه

BORG AL ARAB TECHNOLOGICAL UNIVERSITY

INTRODUCTION TO C PROGRAMMING ESSENTIALS

First Year – Information Technology Program

spring 2026

COURSE INFORMATION

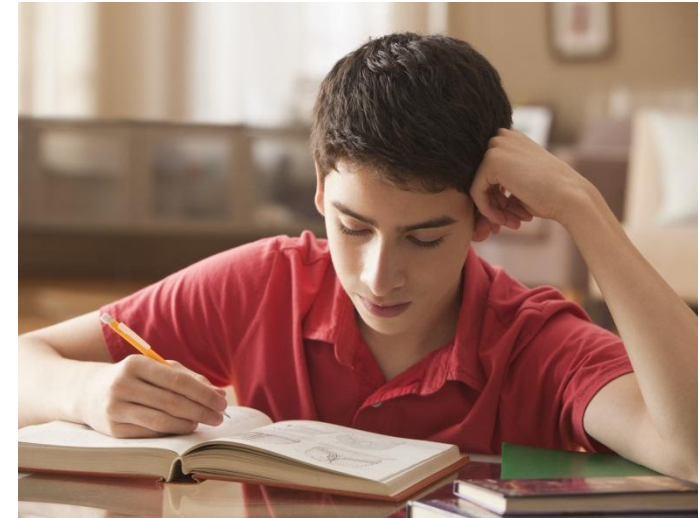
- Lecturer: Dr. Dina Abdelhafiz, Dr. Ghada Fathy
- Credit hours (3)
- Requirements & Grading (Total 150 marks)
 - Class work and attendance (40 marks)
 - Midterm exam (35 marks)
 - Final Exam during finals week (75 marks)
 - Final Exam in hours (3 hours)

ABOUT EXAMS

- All material included in the lectures and uploaded to LMS.
- Notes taken from lecturer at lecture time.
- All material included in the section by your Teaching assistant (TA).

WHAT YOU NEED TO PASS THE EXAMS

Just attend the lectures, Labs, take notes and study





LECTURE I

C LANGUAGE

BASICS



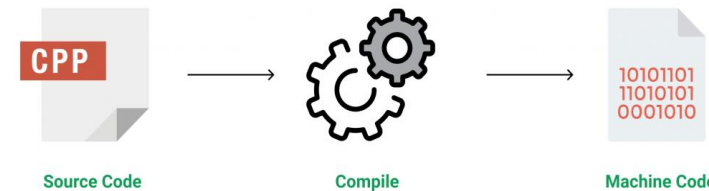
WHAT IS C LANGUAGE?

- C is a general-purpose programming language that has been widely used for over 50 years.
 - It is a fundamental language in the field of computer science.
 - C is closely connected to UNIX, because much of UNIX was written in C.
-

WHAT IS C USED FOR?

C is often used to create programs that need to run fast and work closely with the computer.

- Operating systems, such as parts of **Windows, Linux, and macOS**
- Programs inside devices like **cars, TVs, and home electronics**
- Software that needs to be fast, including **databases and system tools**
- Game engines and programs that **handle graphics**
- Core libraries that **other programming languages use**



WHY C IS IMPORTANT?

Reason	Explanation
Fast	High performance execution
Memory Efficient	Low memory consumption
Portable	Runs on many systems
Foundation	Base for many languages
System-level	Used for OS and hardware

WHAT YOU'LL LEARN TODAY

- How C programs are structured
- Variables, data types, and operators
- Print statement

INSTALL C

If you want to run C programs on your own computer, you need two things:

- A text editor to write C code
- A compiler to turn the C code into a program the computer can run
- An IDE (Integrated Development Environment) is a program that helps you write, compile, and run code.
- Popular IDEs include Code::Eclipse, and Visual Studio.

SYNTAX

```
#include <stdio.h>
```

```
int main() {  
    printf("Hello World!");  
    return 0;  
}
```

SYNTAX

- `#include <stdio.h>`
 - `#` tells C to include a **header file**. This header lets us use **input/output** functions such as `printf()` and `scanf()`.
 - `stdio.h`: **stander input output header**
- `main()` is a special function.
 - Your program starts running here (**entry point**). Any code inside the curly brackets `{}` will be executed.
- `printf()` is a function used to **output (print) text to** the screen. In our example, it prints Hello World!.
- Every C statement ends with a semicolon `;`
- `return 0` **ends the main() function** and **sends a value back** to the operating system.
- Returning 0 usually means "everything worked".

C STATEMENTS

- A **computer program** is a list of instructions that a computer follows.
- In C, these instructions are called **statements**.
- The following statement tells the program to print the text Hello World! to the screen:

```
printf("Hello World!");
```

COMPILETIME ERROR

- Every statement in C must end with a **semicolon ;**
- If you forget the semicolon, the compiler will show an error and the program will not run:

```
#include <stdio.h>

int main() {
    printf("Hello World!")
    return 0;
}
```

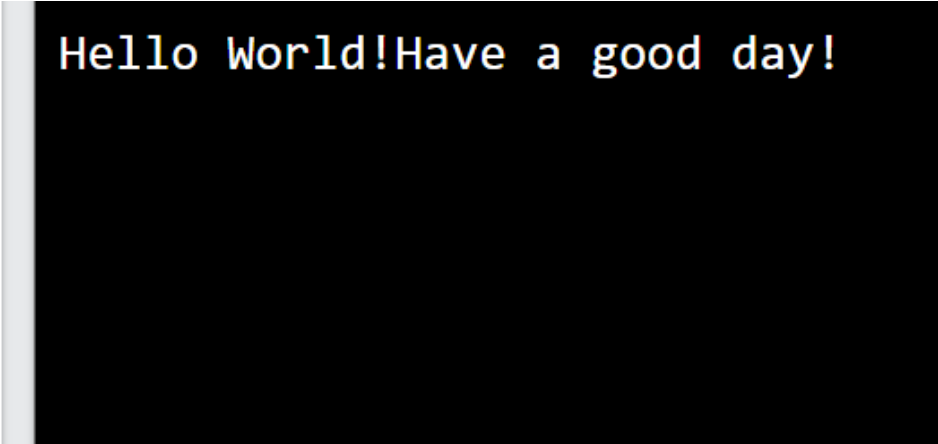
```
In function 'main':
error: expected ';' before 'return'
```

MANY STATEMENTS

- Most C programs contain more than one statement.
- Statements are **executed one by one**, in the same order as they are written:

```
#include <stdio.h>

int main() {
    printf("Hello World!");
    printf("Have a good day!");
    return 0;
}
```

A screenshot of a terminal window with a black background. The text "Hello World!Have a good day!" is displayed in a light blue or cyan monospaced font. There is no space between "Hello World!" and "Have a good day!".

Hello World!Have a good day!

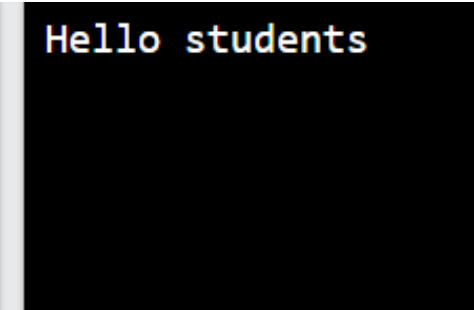
- **Remember:** Each statement is one instruction, and it must end with a semicolon.

OUTPUT VALUES OR PRINT VALUES

- To output values or print text in C, you can use the **printf()** function:
- When you want to print text, the text must be wrapped in double quotes "".

```
#include <stdio.h>

int main() {
    printf("Hello students");
    return 0;
}
```

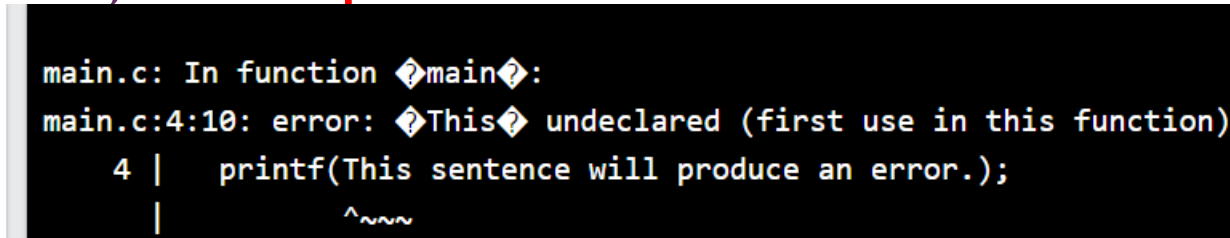
A terminal window with a black background and white text displaying the output of the printf function: "Hello students".

Hello students

- If you forget the double quotes, a **compile time error** occurs:

```
#include <stdio.h>

int main() {
    printf(This sentence will produce an error.);
    return 0;
}
```

A terminal window showing a compile time error. The text is as follows:

```
main.c: In function ?main?:
main.c:4:10: error: ?This? undeclared (first use in this function)
  4 |     printf(This sentence will produce an error.);
    |               ^~~~~
```

The error message indicates that the variable 'This' is undeclared and used in the printf statement.

ESCAPE SEQUENCE

Escape Sequence	Description
\t	Inserts a horizontal tab
\n	Inserts new line
\\	Inserts a backslash character (\)
\"	Inserts a double quote character

```
#include <stdio.h>

int main() {
    printf("\tHello World!\\");
    printf("I am learning C\" \n I am having fun.");
    return 0;
}
```

```
    Hello World!\I am learning C"
    I am having fun.
```

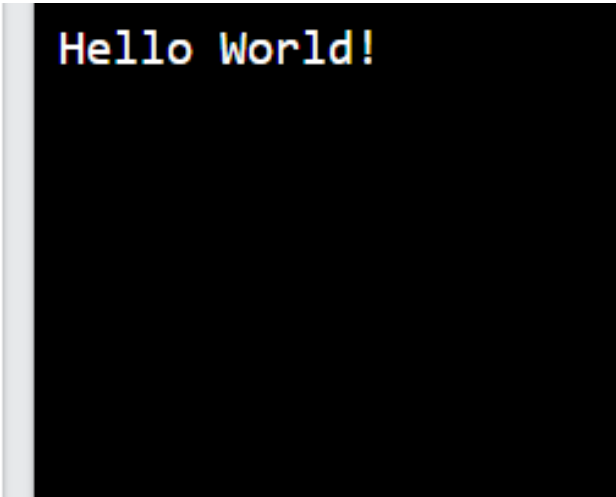
C COMMENTS

- Comments are notes for humans. They help explain code and make it easier to read.
- Single-line comments start with two forward slashes (//).
- Multi-line comments start with /* and end with */.
- Any text between /* and */ will be ignored by the compiler.

SINGLE LINE AND MULTI-LINE COMMENTS

```
#include <stdio.h>

int main() {
    /* The code below will print the words Hello World!
    to the screen */
    printf("Hello World!"); // this comment is ignored!
    return 0;
}
```

A terminal window with a black background and yellow text displaying the output of the C program.

Hello World!

STRUCTURE OF A C PROGRAM

Part	Purpose	Example
Preprocessor	Import libraries	<code>#include <stdio.h></code>
Main function	Program start	<code>int main()</code>
Variables	Store data	<code>int x = 5;</code>
Statements	Program logic	<code>printf();</code>
Return	End program	<code>return 0;</code>

DATA TYPES OR VARIABLES IN C

- Variables are containers for storing data values, like numbers and characters.
- You can think of a variable as a named box where you keep a value that can be used later.

Type	Meaning	Example	Output
int	Whole numbers	int a=10;	10
float	Decimal	float x=3.5;	3.500000
char	Character	char g='A';	A

C THE SIZEOF OPERATOR

- The memory size refers to how much space a type occupies in the computer's memory.
- To actually get the size (in bytes) of a data type or variable, use the sizeof operator:

Data Type	Size
int	2 or 4 bytes
float	4 bytes
double	8 bytes
char	1 byte

```
#include <stdio.h>

int main() {
    int myInt;
    float myFloat;
    double myDouble;
    char myChar;

    printf("%zu\n", sizeof(myInt));
    printf("%zu\n", sizeof(myFloat));
    printf("%zu\n", sizeof(myDouble));
    printf("%zu\n", sizeof(myChar));

    return 0;
}
```

DECLARING (CREATING) VARIABLES

- `// Declare a variable`
`int myNum;`

`// Assign a value later`
`myNum = 15;`

- **Tip:** Variable names should be meaningful, so your code is easier to understand.

C FORMAT SPECIFIERS

- Format specifiers are used together with the `printf()` function to print variables.
- You can think of a format specifier as a **placeholder** that tells C what kind of value will be printed.
- A format specifier always starts with a percentage sign `%`, followed by a letter.

Specifier	Used For	Example	Output
<code>%d</code>	Integer	<code>printf("%d",5);</code>	5
<code>%f</code>	Float	<code>printf("%f",3.2);</code>	3.200000
<code>%c</code>	Char	<code>printf("%c",'A');</code>	A
<code>%s</code>	String	<code>printf("%s","Hi");</code>	Hi

EXAMPLE

```
#include <stdio.h>

int main() {
    // Create variables
    int myNum = 15;           // Integer (whole number)
    float myFloatNum = 5.99; // Floating point number
    char myLetter = 'D';     // Character

    // Print variables
    printf("%d\n", myNum);
    printf("%f\n", myFloatNum);
    printf("%c\n", myLetter);
    return 0;
}
```

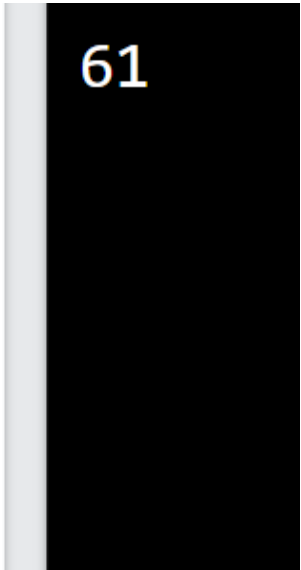
```
15
5.990000
D
```

C DECLARE MULTIPLE VARIABLES

- **Note:** All variables must be of the same type. You cannot mix types in the same declaration.
- **Tip:** Declaring multiple variables on one line is convenient, but declaring them on separate lines can make your code easier to read.

```
#include <stdio.h>

int main() {
    int x = 5, y = 6, z = 50;
    printf("%d", x + y + z);
    return 0;
}
```

A terminal window with a black background and a light gray vertical scrollbar on the left. The number 61 is displayed in white text at the top of the window.

61

C VARIABLE NAMES (IDENTIFIERS)

- All C **variables** must be **identified** with **unique names**.
- These unique names are called **identifiers**.
- Identifiers can be short names (like x and y) or more descriptive names (age, sum, totalVolume).

The **general rules** for naming variables are:

- Names can contain letters, digits and underscores
- Names must begin with a letter or an underscore (`_`)
- Names are case-sensitive (`myVar` and `myvar` are different variables)
- Names cannot contain whitespaces or special characters like `!`, `#`, `%`, etc.
- Reserved words (such as `int`) cannot be used as names



Happy Coding!